Objects II — Methods, Aliasing, and Collections

Lecture 29

CS 1001

Kirk Duran

Today

- Define **methods** as behavior associated with a class.
- Explain how `self` identifies the particular instance involved in a method call.
- Distinguish methods that return information from methods that mutate object state.
- Extend the reference and aliasing model to user-defined objects.
- Define value equality with `__eq__` and distinguish it from identity.
- Customize object descriptions with `__str__` and `__repr__`.
- Store objects inside lists and dictionaries and iterate over those collections.
- Build objects that themselves contain collections or other objects.

Key idea

Methods operate on object state, while references determine *which* object a method call or mutation affects.

Where We Left Off

Lecture 28

- class and instance
- `__init__`
- attributes
- object state
- references to instances

Lecture 29

- behavior through methods
- deeper meaning of `self`
- mutation and aliasing
- identity and custom equality
- readable object descriptions
- collections of objects
- nested compound structures

Part 1: From Object State to Object Behavior

Objects Can Represent Both State and Behavior

State

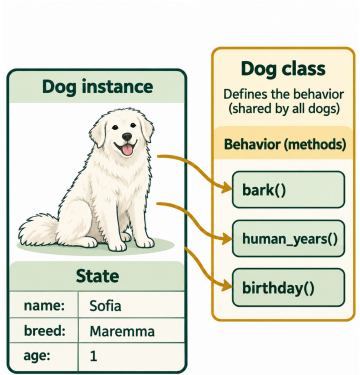
A `Dog` instance might store:

- `name`
- `breed`
- `age`

Behavior

A `Dog` might also be able to:

- report a description
- compute a value from its state
- change its state



Definition: Method

Definition

A **method** is a function defined as part of a class. A method can use the state of a particular instance and can optionally modify that state.

Example

```
class Dog:
   def __init__(self, name:  str, breed:  str, age:  int):
      self.name = name
      self.breed = breed
      self.age = age
   def bark(self) -> str:
      return self.name + " says bark!"
```

- Its first parameter is conventionally named `self`.
- It is called using dot notation on an instance.

Calling a Method Uses Dot Notation

Class and call

```
dog1 = Dog("Sofia", "Maremma", 1)

message = dog1.bark()
```

- `dog1` refers to one particular `Dog` instance.
- `.bark` selects behavior defined by the `Dog` class.
- The method executes using `dog1` as the current instance.
- The returned string is bound to `message`.

Key idea

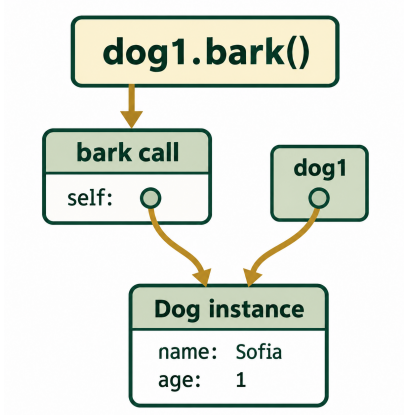
Dot notation connects a method definition to the instance on which that behavior should operate.

What Does `self` Mean?

Definition

Inside an instance method, `self` is the parameter that refers to the instance involved in the current method call.

- `self.name` means the `name` attribute of that instance.
- `self.age` means the age attribute of that instance.
- Different calls can bind `self` to different instances.



Trace: `self` Is Bound for the Call

Program

```
dog1 = Dog("Sofia", "Maremma", 1)
dog2 = Dog("Buddy", "Labrador", 3)

a = dog1.bark()
b = dog2.bark()
```

Call	`self` refers to	attribute read
`dog1.bark()`	`dog1` instance	`dog1.name`
`dog2.bark()`	`dog2` instance	`dog2.name`

Methods Can Accept Additional Arguments

Add this method inside the same Dog class

```
def rename(self, new_name:  str) -> None:
   self.name = new_name
```

Calling:

Call

```
dog1.rename("Luna")
```

conceptually binds:

- `self` → the instance referred to by `dog1`
- `new_name` → `"Luna"`

Part 2: Query Methods and Mutating Methods

Some Methods Compute Information Without Changing State

Add this query method inside Dog

```
def human_years(self) -> int:
   return self.age * 7
```

- The method reads `self.age`.
- It computes and returns a new value.
- The object's attributes are unchanged.

Key idea

Reading object state does not imply mutating object state.

Other Methods Intentionally Change State

Add this mutating method inside Dog

```
def birthday(self) -> None:
   self.age = self.age + 1
```

- The right-hand side reads the current age.
- Assignment writes a new value into the instance's age attribute.
- The change remains after the method returns.
- The useful result is the changed object state, so no explicit value needs to be returned.

Query Versus Mutation Is an Interface Decision

Method style	Primary effect	Typical return
query	reads state	computed value
mutation	changes state	often `None`

Examples

`dog1.human_years()` → returns information
`dog1.birthday()` → changes `dog1.age`

Key idea

A method's contract should make it clear whether callers should expect a returned value, a state change, or both.

Trace the State Change

Before

`dog1` → `Dog(name="Sofia", age=1)`

Call

```
dog1.birthday()
```

After

`dog1` → `Dog(name="Sofia", age=2)`

- The variable `dog1` was not rebound.
- The same instance still exists.
- One attribute inside that instance changed.

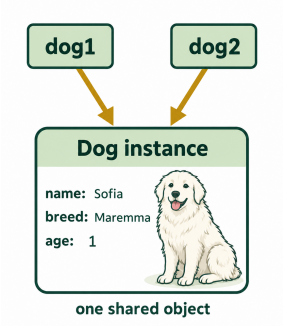
Part 3: References and Object Aliasing

Assignment Still Does Not Automatically Copy

Program

```
dog1 = Dog("Sofia", "Maremma", 1)
dog2 = dog1
```

Assignment copies the reference; it does not construct another Dog.



Key idea

Two names are aliases when they refer to the same object.

Mutation Through Either Alias Reaches the Same Instance

Program

```
dog1 = Dog("Sofia", "Maremma", 1)
dog2 = dog1
dog2.birthday()
```

After the call

`dog1.age` $\rightarrow$ 2
`dog2.age` $\rightarrow$ 2

- The method call occurred through the name `dog2`.
- The changed state belongs to the shared instance, not to a variable name.

Constructing Again Produces an Independent Instance

Program

```
dog1 = Dog("Sofia", "Maremma", 1)
dog2 = Dog("Sofia", "Maremma", 1)
```

- The attribute values happen to match.
- Two constructor calls produce two distinct instances.
- Mutating one does not mutate the other.

dog1 $\rightarrow$ Dog instance A

dog2 $\rightarrow$ Dog instance B

Identity Asks Whether Two References Reach the Same Object

Example

```
dog1 = Dog("Sofia", "Maremma", 1)
dog2 = dog1
dog3 = Dog("Sofia", "Maremma", 1)
```

Expression	Result
`dog1 is dog2`	`True`
`dog1 is dog3`	`False`

Key idea

`is` asks about identity: are these two references to the exact same instance?

Identity and Equality Answer Different Questions

Identity

`a is b`
Are `a` and `b` references to the same object?

Equality

`a == b`
Should these two values be considered equivalent?

- Identity is built into the reference model.
- A class can define its own notion of value equality.
- So far, our `Dog` class has no custom equality rule: distinct instances compare unequal with `==`, even if their attributes match.

Passing an Object to a Function Can Also Create an Alias

Function

```
def celebrate(dog:  Dog) -> None:
   dog.birthday()
```

Call

```
dog1 = Dog("Sofia", "Maremma", 1)
celebrate(dog1)
```

During the call, the parameter `dog` and the caller's name `dog1` refer to the same instance.

Key idea

Parameter binding does not automatically copy mutable objects.

Part 4: Equality and Object Descriptions

Special Methods Connect Objects to Python Operations

Definition

A **special method** has a name surrounded by double underscores. Python uses these methods to support familiar operations on our objects.

Operation	Method	Purpose
`a == b`	`__eq__(self, other)`	value equality
`str(a)`, `print(a)`	`__str__(self)`	readable description
`repr(a)`	`__repr__(self)`	debugging description

Key idea

We define the methods inside the class; callers normally use the corresponding Python operations. These examples read state without changing it.

Define Value Equality with `__eq__`

For this example, two Dogs are equal when name, breed, and age all match.

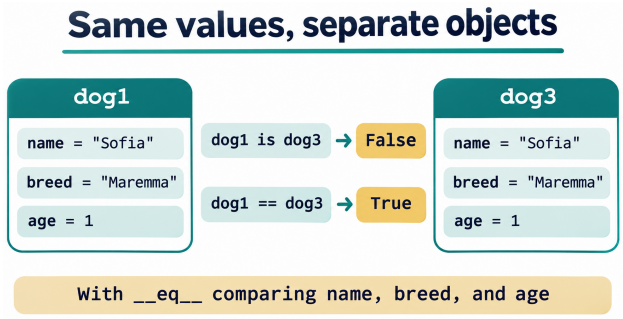
Add inside the existing Dog class

```
def __eq__(self, other:  object):
   if not isinstance(other, Dog):
      return NotImplemented
   return (self.name == other.name
         and self.breed == other.breed
         and self.age == other.age)
```

- `self` is one operand; `other` is the value being compared.
- Check the type before reading `other`'s attributes.
- `NotImplemented` lets Python try the other operand's comparison. If neither supports equality, Python falls back to identity.
- This is a chosen rule for this model, not proof that two records represent the same real dog.

Equal Values Can Still Be Separate Objects

After adding `__eq__`, construct two Dogs with matching attributes:



Key idea

`__eq__` changes the meaning of `==`; it does not change `is`, copy objects, or merge their state.

Make Objects Readable with `__str__`

Add inside Dog

```
def __str__(self) -> str:
   return f"{self.name} ({self.breed}), age {self.age}"
```

Calls and output

```
dog1 = Dog("Sofia", "Maremma", 1)
text = str(dog1)
print(dog1)
```

Output: `Sofia (Maremma), age 1`

- `str(dog1)` returns the description as a string.
- `print(dog1)` uses that description and displays it.
- `__str__` must **return a string**; printing inside the method is not a substitute.

Expose Useful Details with `__repr__`

Add inside Dog

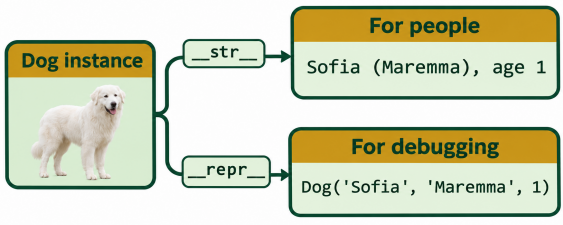
```
def __repr__(self) -> str:
   return (f"Dog({self.name!r}, {self.breed!r}, "
        f"{self.age!r})")
```

Call and returned string contents

`repr(dog1)` → `Dog('Sofia', 'Maremma', 1)`

- Aim for a precise description useful while debugging.
- The f-string conversion `!r` uses `repr` on each attribute, so strings include quotes and escapes.
- Constructor-like output is useful when practical, but not required.
- This method also returns a string; it does not construct another Dog.

One Object, Two Descriptions



Containers use their elements' debugging descriptions

`print([dog1])`
Output: `[Dog('Sofia', 'Maremma', 1)]`

`print(dog1)` uses `__str__`; the list uses each element's `__repr__`.

What Happens If a Description Method Is Missing?

- With both methods: `str` uses `__str__`; `repr` uses `__repr__`.
- With only `__repr__`: the inherited `__str__` falls back to that representation too.
- With only `__str__`: `repr` still uses the default object representation.
- With neither: this simple class gets a default representation such as `<__main__.Dog object at 0x...>`.

Key idea

A readable description does not define equality. Two objects can print the same text and still compare unequal unless the class defines an appropriate equality rule.

Predict: Equality After Mutation

Use Dog with all three special methods, plus `birthday` from earlier.

Program

```
a = Dog("Sofia", "Maremma", 1)
b = Dog("Sofia", "Maremma", 1)
c = a
b.birthday()
```

In class

Predict `a == b`, `a is c`, `a == c`, and the output of `print([a, b])`.

Reveal: Equality Reads the Current State

Expression	Result	Reason
`a == b`	`False`	ages now differ
`a is c`	`True`	same instance
`a == c`	`True`	same current attribute values

Output of `print([a, b])`

```
[Dog('Sofia', 'Maremma', 1), Dog('Sofia', 'Maremma', 2)]
```

Key idea

Mutating `b` changes its value comparison with `a`. The alias `c` still reaches `a`; neither comparison nor formatting copies an object.

Part 5: Collections of Objects

Define the Student Used in These Collections

A class for the next examples

```
class Student:
   def __init__(self, name:  str, age:  int, gpa:  float):
      self.name = name
      self.age = age
      self.gpa = gpa
```

- This version stores name, age, and GPA; Lecture 28's design stored a major instead of age.
- Read constructor arguments using the definition in the current example.

A List Can Contain References to Objects

Example

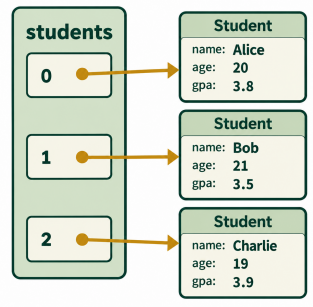
```
students = [
   Student("Alice", 20, 3.8),
   Student("Bob", 21, 3.5),
   Student("Charlie", 19, 3.9)
]
```

- The list is one collection value.
- Each element refers to a `Student` instance.
- Each instance carries its own compound state.

Mental Model: Collection of References

List positions

```
students[0]
students[1]
students[2]
```



Key idea

The list organizes references; the objects contain the student-specific state.

Iterating Over a List of Objects

Example

```
for student in students:
   print(student.name)
```

Each iteration:

1. binds `student` to the next list element;
2. that element is a reference to a `Student` instance;
3. dot notation reads an attribute of that instance.

Sequence Patterns Still Apply to Objects

Suppose we want the names of students with GPA at least 3.5.

Explicit loop

```
names = []
for student in students:
   if student.gpa >= 3.5:
      names.append(student.name)
```

Key idea

Earlier loop patterns do not disappear. The values being processed are now compound objects rather than primitive values.

A Dictionary Can Also Store Objects

Example

```
students_by_id = {
   101:  Student("Alice", 20, 3.8),
   205:  Student("Bob", 21, 3.5)
}
```

Then:

Access

```
students_by_id[205].name
```

- The dictionary associates an ID key with an object reference.
- Dot notation continues from the selected object.

Aliasing Can Occur Through a Collection

Program

```
alice = Student("Alice", 20, 3.8)
students = [alice]
students[0].gpa = 4.0
```

After the mutation:

Observation

`alice.gpa` $\rightarrow$ `4.0`

- `alice` and `students[0]` both reach the same instance.
- The list does not contain an independent copy of the student.

Part 6: Objects Containing Structured Data

An Object Attribute Can Itself Be a Collection

Example

```
class Classroom:
   def __init__(self, name:  str):
      self.name = name
      self.students = []
```

- `name` refers to a string.
- `students` refers to a mutable list.
- The object's state can therefore include structured values.

Methods Can Mutate Collection Attributes

Add this method inside Classroom

```
def add_student(self, student:  Student) -> None:
   self.students.append(student)
```

Use

```
room = Classroom("180-0114")
alice = Student("Alice", 20, 3.8)
room.add_student(alice)
```

Key idea

A mutating method may change an attribute by mutating the collection that attribute refers to. Appending an object stores a reference to that same object; it does not create an automatic copy.

An Object Can Also Contain Another Object

Example

```
class Address:
   def __init__(self, street:  str, city:  str):
      self.street = street
      self.city = city
```

Person contains an Address reference

```
class Person:
   def __init__(self, name:  str, address:  Address):
      self.name = name
      self.address = address
```

Nested Attribute Access Is Evaluated Left to Right

Example

```
addr = Address("123 Main St", "San Luis Obispo")
person = Person("Alice", addr)

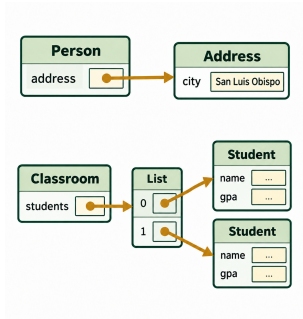
person.address.city
```

Evaluate in stages:

1. `person.address` produces the referenced `Address` object.
2. `.city` then reads that object's `city` attribute.

Nested Objects Extend the Same Reference Model

- A variable can refer to an object.
- An attribute can refer to another object.
- A list element can refer to an object.
- A dictionary value can refer to an object.



Key idea

The syntax changes, but the fundamental model remains names and containers holding references to values.

Part 7: Integrated Tracing

Integrated Example

For this final trace, use a fresh, smaller Student definition (name and GPA only).

Classes

```
class Student:
   def __init__(self, name:  str, gpa:  float):
      self.name = name
      self.gpa = gpa
   def set_gpa(self, gpa:  float) -> None:
      self.gpa = gpa
```

Predict Before Tracing

Program

```
a = Student("Alice", 3.5)
b = a
students = [a]
students[0].set_gpa(3.9)
```

Predict:

1. How many `Student` instances exist?
2. What does `a.gpa` evaluate to?
3. What does `b.gpa` evaluate to?
4. Is `a is students[0]` true or false?

Reveal: One Shared Instance

Only one constructor call occurred, so there is one `Student` instance.

Three references reach this one Student

`a`, `b`, `students[0]` $\longrightarrow$ `name="Alice", gpa=3.9`

Therefore:

- `a.gpa` $\rightarrow$ `3.9`
- `b.gpa` $\rightarrow$ `3.9`
- `a is students[0]` $\rightarrow$ `True`

A Useful Tracing Procedure for Objects

When object code becomes difficult, trace these questions in order:

1. Which instances have actually been constructed?
2. Which names or container positions refer to each instance?
3. Which instance is bound to `self` for the current method call?
4. Which attributes are read?
5. Which attributes or contained mutable values are changed?
6. What value, if any, is returned?

Key idea

Separate **control flow**, **references**, and **object state**; do not mentally collapse them into one step.

Three Common Wrong Models

Names own objects

`b = a` does **not** construct a second object. It creates another reference to the same object.

Collections copy objects

Putting `alice` into a list does **not** automatically create an independent student.

`self` is global

`self` is a method parameter whose binding is established separately for each call.

What We Are Not Adding Yet

- inheritance and class hierarchies
- private attributes or property decorators
- class variables and static methods
- operator customization beyond equality
- abstract classes or interfaces
- advanced object-oriented design patterns

Key idea

The goal is a precise mental model of instances, methods, references, and compound structures before adding larger object-oriented abstractions.

Takeaways

1. A method is behavior defined as part of a class.
2. `self` refers to the particular instance involved in the current method call.
3. Methods may query state, mutate state, or deliberately do both.
4. Assignment and parameter binding can create aliases to the same object.
5. `is` asks about identity; `__eq__` defines value equality for `==`.
6. `__str__` gives a readable description; `__repr__` gives debugging details.
7. Collections can store references to objects, and objects can contain collections or other objects.
8. The same reference model developed for lists now explains compound object structures.

Lab connection

Lab Programming 3 will ask you to develop functions that operate on objects using these same state, reference, and interface ideas.